of placing parentheses immediately after the expression (with the caveat that the entire declaration must also be wrapped by parentheses) like this: `(function () { // function body })();`. This technique of immediate execution is known as a *self-invoking* (or *self-calling*) function:

```
(function (string $txt): void {
    print $txt . PHP_EOL;
})("Go go go!");
```

```
Output:
Go go go!
```

Like any expression, anonymous function declarations can be assigned to variables, simply using standard variable assignment:

```
$anon = function (): void {
    print "Who am I?" . PHP_EOL;
};
$anon();
```

```
Output:
Who am I?
```

This looks very similar to the syntax for variable functions in the previous section, but there's a subtle difference. `$anon` is a direct reference to a function, rather than a string which evaluates to a function name. There's no need for PHP to evaluate a string then check it matches a known function name, and no danger of triggering a fatal error because the string doesn't match a declared identifier.

## Passing Functions As Data

If you've been paying attention so far you have probably already figured out that the variable used to reference (and call) an anonymous function could also be the argument of *another* function declaration, like so:

```
handleCallback(function (): void {
    print "Who am I?" . PHP_EOL;
});
```